

Adaptive Merkle Trie Versioning for Efficient Structured Data Management

Atharva Sheersh Pandey, Adhyan Jain, Poornima Nedunchezian

atharva.sheersh2024@vitstudent.ac.in; adhyan.jain2024@vitstudent.ac.in

Corresponding author: Poornima N (poornima.n@vit.ac.in)

School of Computer Science and Engineering

VIT Vellore

Abstract: Revon is a Python prototype for durable, linear-history key/value versioning. It combines a fixed-depth Merkle hash trie, addressed changesets and an operation-count policy that selects log aggregation or hash-pruned comparison for version differences. We validate the repaired JSON ownership and identity implementation on a separate Ubuntu cloud machine using 516 isolated workflow executions, including synthetic histories, public taxi records and threshold controls. Across eight synthetic scenarios and one 100,000-record public scenario, seven paired trials favour the hybrid variant over forced Merkle in eight diff-latency intervals; the dense case includes parity. A frozen 16,384-operation policy selects Merkle at 32,768 operations even though forced log is faster in every matched trial, so the evidence does not establish an optimal threshold or superiority over always-log. A separate 72-execution supplement verifies process telemetry and fixed-trie geometry. Million-record runs establish correctness and feasibility for one trial per system, without a comparative performance claim. Results concern complete Revon API and Dolt CLI workflows, rather than isolated storage engines. The contribution is a reproducible comparison of two correct diff paths over the same durable representation, with explicit costs and limits.

Keywords: data versioning, Merkle hash trie, hybrid differencing, content-addressed storage, benchmark methodology

1 Introduction

Structured-data version control has to do more than archive files. It must preserve exact versions, apply sparse updates without rebuilding the whole state, restore earlier versions, and compare states efficiently. Git offers a useful starting point: immutable objects named by their content, linked through parent commits [1]. Its objects are organized around files and directories, however. Record stores have a different shape: keys may remain stable while update density changes, and two versions may differ by a few edits or by a long commit history.

Revon asks whether a fixed-depth Merkle hash trie can combine incremental structural sharing with an operation index. Revon-H adapts only its diff path: it selects log aggregation for eligible intervals within an operation threshold and Merkle comparison otherwise. The trie geometry remains fixed. SQLite supplies atomic, durable storage. The title's efficiency claim is restricted to the measured operations and workloads; it does not assert that all structured-data management becomes cheaper.

We use Dolt as the production comparator because it combines SQL, a Git-style commit graph, and content-addressed Prolly Trees [2], [3]. Revon is a smaller research prototype,

not a feature-for-feature replacement for Dolt. Our results compare Revon’s in-process Python API with a Dolt CLI workflow; they do not isolate the performance of either tree structure.

Research Questions

RQ1: How do Revon-H’s latency and storage compare with Snapshot, Log-only, and Dolt across dataset sizes, mutation densities, and update localities?

RQ2: Does choosing between log and Merkle diff improve latency over forced-Merkle Revon-M while preserving correctness?

RQ3: How do branching factor and trie depth affect latency, storage, and the amount of work examined?

Contributions

We implement a canonical SHA-256-addressed, fixed-depth trie with incremental rewriting of paths affected by multi-key updates.

We design a hybrid diff policy that combines addressed changesets with hash-pruned tree comparison.

We build an atomic SQLite repository for objects, commits, aliases, measurements, and HEAD, and verify object hashes after reopening it.

We compare five systems through six import/diff workflows on deterministic histories, retain per-trial semantic and historical-state checks, balance execution order, and validate the repaired source on a separate Linux cloud machine.

We measure how branching factor and depth affect the fixed trie, rather than assuming that $b=8$ and $d=4$ are universally best.

2 Related Work

Content Addressing and Persistent Structures

Merkle’s authenticated-tree construction showed that a compact root digest can commit to a larger collection [4]. Git and IPFS apply content addressing to software history and networked content [1], [6]. Persistent-data-structure theory formalized path copying and access to earlier states [5]. Later persistent dictionaries explore ordered, disk-oriented alternatives: stratified B-trees [16] and fully persistent B-trees [19]. Revon instead routes hashed keys through a fixed-depth trie, trading ordered range access for hash-pruned comparison. Scientific array systems such as SciDB and TimeArr preserve versions using array-specific storage layouts and deltas [17], [18]. TardisDB integrates version access with a main-memory database, while Saquib et al. study replicated versioned structures across wide-area systems [20], [21]. These systems illuminate adjacent design choices, but they do not compare Revon’s log and Merkle diff paths over the same history.

Merkle Search Trees combine search-tree ordering with Merkle authentication to reconcile state [7]. Revon also prunes comparisons by hash, but uses fixed-depth routing and a centralized linear history rather than a replicated CRDT. LBFS uses content-defined chunking for redundancy detection [8]; Dolt and Noms use related boundary strategies.

Ordered indexes preserve range operations, while content-defined boundaries can improve deduplication when data shifts. Revon’s current benchmark measures neither capability.

Structured Dataset Versioning

Dataset-versioning research covers collaboration, storage layout, and retrieval. DataHub studies collaborative dataset history at scale [9]. Decibel brings branching into a relational engine and compares version-first, tuple-first, and hybrid layouts [10]. OrpheusDB adds versioning to a conventional DBMS and optimizes version retrieval through partitioning [11]. Bhattacharjee et al. formalize the storage-versus-reconstruction trade-off [12], while Delta Lake uses transaction logs and checkpoints for ACID tables [13]. More recent work studies how to choose materialized versions and deltas over a version graph, subject to storage and retrieval costs [26]. A FAIR-compliant proposal focuses on identifying and describing dataset releases consistently [27]; it complements storage research but does not provide a diff algorithm. Revon addresses a narrower question with a key/value prototype and does not solve these broader dataset-management problems.

System Design Gap

Noms stores structured values in a Merkle DAG and supports diff and synchronization [14]. ForkBase combines content addressing with fork and deduplication semantics [15]. Dolt’s content-addressed Prolly Trees combine ordered B-tree-like indexing, structural sharing, and native diff [2], [3]. Git4Data is another nearby direction: it presents SQL-level snapshot, branch, diff, and merge operations over a relational engine [29]. The available Git4Data manuscript is an arXiv preprint, so we do not treat its results here as peer-reviewed or independently reproduced evidence. Revon has a narrower scope: it is a single-writer, linear-history key/value prototype with fixed-depth hash routing, not a SQL database or branching system.

Table 1 summarizes the systems and studies most closely related to the implemented design.

Table 1 Related-work summary

System or study	Representation or focus	Relevance to Revon
Persistent DS [5]	Path copying	Formal structural-sharing basis
Git [1]	Object DAG	Identity and commit ancestry; file-oriented
IPFS [6]	Merkle DAG	General content-addressed versioning
MST [7]	Merkle search tree	Authenticated ordered reconciliation
DataHub [9]	Version graph	Collaborative dataset motivation
Decibel [10]	Relational layouts	Native database branching
OrpheusDB [11]	Partitioned versions	Storage/retrieval optimization
Delta Lake [13]	Log/checkpoints	ACID time travel at data-lake scale
Noms [14]	Merkle DAG	Structured content addressing
ForkBase [15]	Forkable CAS	Fork and deduplication semantics
DoltHub [2,3]	Prolly Trees	Mature production comparator

Design Implications and Research Positioning

Versioning systems face three recurring costs: retaining history, creating a version, and later reconstructing or comparing versions. Snapshots simplify retrieval by copying unchanged records. Logs defer work until replay or checkpointing. Path copying creates only changed paths and shares the rest [5], [12]. Decibel and OrpheusDB show that physical layout affects both version creation and retrieval [10], [11]; graph-based analyses make the same storage/retrieval trade-off explicit [26]. Reproducible-research work addresses a separate problem: preserving builds and artifacts for later inspection does not show that one storage algorithm is faster [24]. For that reason, Revon reports import, commit, checkout, diff, and storage measurements separately.

Different research areas also give “change” different meanings. Differential dataflow incrementally recomputes query results as timestamped inputs change [23]. Data-provenance research asks why and where output facts were derived [22], and DSLog stores and queries fine-grained array lineage [28]. Explain-Da-V uses data transformations to explain semantic dataset changes [25]. Baseline uses operation-based differencing that can account for refactorings and schema changes [30]. Revon reports a more limited result: which keys were added, removed, or changed in value between two states. It does not infer causes, transformation lineage, query impact, or schema intent. These systems offer useful context, but their outputs and workloads cannot be compared directly with Revon’s key/value diff timings.

Revon’s contribution is a controlled comparison of two correct diff paths over the same durable history, selected using properties of the requested interval. Each Revon-H commit

stores both a Merkle root and an addressed changeset, so the choice of diff path does not change version semantics. We hypothesize that log aggregation suits short, sparse intervals and that hash pruning may help after longer operation sequences. A crossover must be measured rather than inferred from the switching rule. Work such as Git4Data also shows that database-native version control remains an active area [29], which narrows our claim: Revon evaluates adaptive differencing for a fixed-depth hash trie; it is not a general-purpose data-versioning platform. Snapshot and Log-only remain representation baselines. The public-record extension covers one release with generated updates. It does not establish performance across diverse datasets, independent hardware, concurrent workloads, or production systems.

3 Revon Design

Figure 1 summarizes the architecture and implemented research scope.

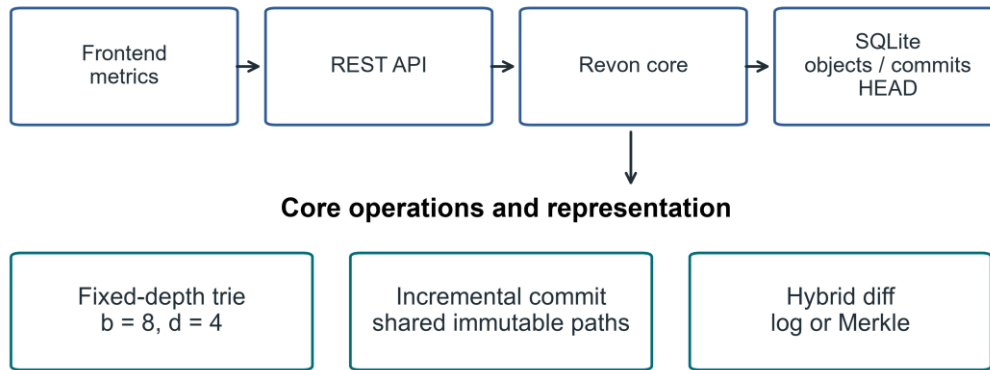


Fig. 1 Revon architecture and the implemented research scope

Addressed Object Model

Revon serializes trie nodes and changesets as canonical JSON and names each object by its SHA-256 digest. A commit digest covers the root, parent, changeset, message, and timestamp. Labels such as v1 and v2 are readable aliases only; they do not affect object identity. A durable HEAD reference identifies the current commit.

Fixed-Depth Merkle Hash Trie

The main experiment uses $b=8$ and $d=4$. At each level, Revon reads three bits from SHA-256(key), so twelve routing bits select among 4,096 possible leaf buckets. Each bucket retains the full key, allowing Revon to distinguish keys with the same routing prefix. With uniform hashes, expected bucket occupancy is N/b^d . We evaluate other b, d pairs separately rather than treating this default as optimal.

Incremental Hashing and Commit

Revon canonicalizes each mutation batch, routes the keys, and groups updates that share trie prefixes. It rewrites each affected leaf once, then rebuilds only the ancestor paths that contain changes. Unchanged child hashes are reused in the new internal nodes, which

preserves structural sharing. A single SQLite transaction writes the new objects, changeset, commit metadata, version alias, measurements, and updated HEAD. On reopen, Revon verifies the stored payload hashes and references.

Hybrid Diff

The log path walks ancestry and aggregates key operations; the Merkle path compares roots recursively, skips equal hashes, and examines entries under unequal paths. Revon-H validates both version identifiers, checks ancestry in either direction, and counts changeset operations. It chooses log aggregation when an ancestor path exists and the count is at most T ; otherwise it uses Merkle comparison. Reverse log requests swap old/new values, and a same-version request returns an empty diff. Forced log mode rejects unrelated versions. Unknown versions raise an error. Revon-M forces Merkle comparison, although its public version-pair API also checks ancestry. Both paths return the same sorted semantic diff. Commit and checkout are separate operations.

The primary Linux workflow matrix retains $T=4,096$, matching the original workflow protocol. A separate Linux control tests the previously frozen candidate $T=16,384$. Bucket capacity b^d and the operation threshold are distinct parameters; neither is tuned on these Linux results.

JSON value ownership and identity

Values are JSON nulls, booleans, finite numbers, strings, lists and objects with string keys. Input containers are validated and detached before retention; read and diagnostic views export detached containers. Immutable scalars can be shared. Unsupported Python-only values, non-string nested object keys, non-finite numbers and cyclic containers are rejected before writing. Equality follows the existing v1 JSON encoding: sorted object keys, compact separators, UTF-8, no NaN. Thus true, 1, 1.0 and signed floating zero retain their encoded distinctions. This is a Python encoding contract, not a claim of compliance with a cross-language canonical-JSON standard.

Algorithm 1 (batch update). Validate and own the input values; reject overlapping puts/deletes and missing deletions before interning. Read each old value and retain only encoded-identity changes. Group effective mutations by routing prefix. Rewrite each touched leaf, sort entries, hash its complete payload and rebuild affected ancestors; reuse untouched child hashes. Record the addressed changeset and commit. The ownership invariant is that callers cannot mutate retained values through supported reads or earlier inputs.

Algorithm 2 (log composition). Traverse the chronological ancestor interval. For each key retain its first old existence/value and its last new existence/value. Discard the key exactly when these endpoint states have equal existence and encoded value. This cancels insert-then-delete and update-then-restore sequences. Null and absence remain distinct. Sort remaining keys; reverse requests swap endpoints and added/deleted labels.

Algorithm 3 (selection). Validate versions and determine ancestry in either direction. Sum changeset operation counts along the interval. Select log when eligible and the count is at most T ; otherwise select Merkle. Merkle skips equal addressed subtrees and compares values in unequal leaves. Both paths apply the same encoded-identity rule and return detached outputs.

Cost model. Let N be records, d depth, b branching factor, m batch keys, h interval length, p accumulated operations, u distinct interval keys and s serialized output bytes. Under uniform routing, expected leaf occupancy is $N/(b^d)$, but worst-case occupancy is N . Batch work includes input validation/copying, m routed old-value lookups, scans of their occupied leaves, sorting and serialization of touched leaves, and copying affected internal child maps. It is not a dataset-independent $O(d)$ bound.

Log composition takes ancestry traversal plus $O(p + u \log u)$, encoded-value comparisons and output copying/serialization. Merkle work depends on visited node pairs, child-slot sorting and examined leaf entries, plus value comparisons and sorted output. Both public version-pair modes perform ancestry work; a reverse query can first traverse to genesis in the unsuccessful direction. Output costs are at least proportional to s . Larger JSON payloads add comparison/copying costs that scalar-only benchmarks do not characterize.

The SQLite repository eagerly loads historical nodes, changesets and commits into memory. It is a durable resident object model; retained history must fit the available memory. Open/verification scales with retained payloads, and no disk-page cache or eviction policy is implemented. Atomic SQLite persistence is distinct from untested production concurrency and operational guarantees.

Figure 2 shows the implemented version-pair decision, including eligibility and reverse handling.

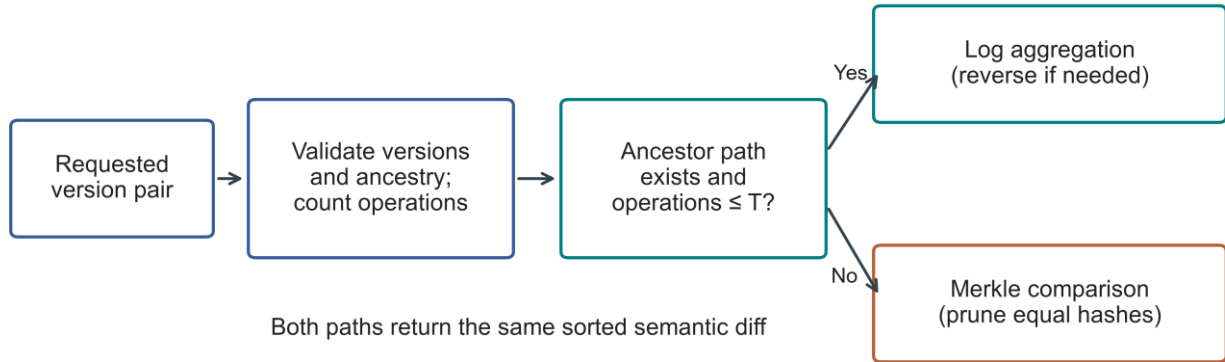


Fig. 2 Actual hybrid selection for a version pair; T denotes the operation threshold. Invalid versions raise an error before selection. Both paths share the persistent trie

4 Experimental Methodology

Environment and evidence boundaries

The current evaluation runs on a separate Google Compute Engine n2-standard-4 VM in us-east1-c. Table 2 records the environment and separates the two source snapshots. Windows and WSL results from earlier revisions remain archived; they are not pooled with the Linux trials or used as repaired-source measurements. The new machine provides an additional hardware environment, not an independent research team or an experiment isolating the effect of Linux.

Table 2 Linux environment and separately retained evidence

Property	Recorded configuration
Machine	GCP n2-standard-4; Intel Cascade Lake; 4 vCPUs; 16 GB RAM; us-east1-c
Operating system	Ubuntu 24.04.5 LTS; x86_64; kernel 7.0.0-1011-gcp
Storage	100 GB balanced persistent disk; no swap configured
Runtime	CPython 3.14.3, Clang 22.1.1 build; GIL enabled; JIT not enabled; SQLite 3.50.4
Comparator and packages	Dolt 2.3.1; psutil 7.1.0; PyArrow 25.0.1
Primary source	linux-validation-20261008: archived repaired source, fixed before measurement
Supplement source	linux-final-supplement-20261008: same core; zero-aware I/O sampler and geometry history checks
Background conditions	Google Ops Agent active; filesystem cache not cleared; monitoring included

Each bundle contains the executed source, hashes, environment records, frozen workload definitions, execution order, raw rows and audits. PYTHONHASHSEED is fixed to 20261008. Source snapshots include the exact working revision rather than relying on the repository HEAD alone. Unit tests passed on the VM: 67 before the primary campaign and 69 before the supplement. An early supplement preflight failed because the transferred test_data directory was absent; the empty fixture directory was created and all tests rerun before any supplement timing began. This packaging failure is retained in the supplement records.

Variants and workloads

Snapshot stores a complete state per version; Log-only retains the initial state and operations; Revon-M forces Merkle diff; Revon-H selects its path; and Dolt uses a primary-key SQL table and native JSON diff. Dolt SQL INSERT and CSV table-import workflows are measured separately, giving six workflow variants for five systems. Subsequent operations have the same semantic contract, but the interfaces and storage features differ. Table 3 defines workload identifiers used in the figures.

Table 3 Linux workloads; all primary histories have ten generated commits

ID / scenario	Rows	Changes / commit	Locality
S1 Small sparse	1,000	10	Spread
S2 Medium sparse	10,000	10	Spread
S3 Medium dense	10,000	1,000	Spread
S4 Large sparse	100,000	100	Spread
S5 Application-key local	100,000	100	Lexically adjacent keys
S6 Range local	100,000	100	Contiguous key intervals
S7 Repeated key	100,000	100	Repeated small key set
S8 Hash-route local	100,000	100	Shared 9-bit hash prefix
P1 TLC public records	100,000	100	Generated fare corrections

The primary synthetic matrix has eight scenarios, six variants, two warm-ups and seven measured trials: 432 executions. The public-record matrix adds 27 executions for Revon-M, Revon-H and Dolt SQL. Two threshold-control workloads add 54 executions. Three million-row diagnostics complete the 516-execution campaign: 114 warm-ups, 399 measured trials and three diagnostics. No failed timing execution is silently replaced or excluded; all 516 completed with verified outputs.

Public dataset and generated histories

We extend the synthetic evaluation with January 2024 yellow-taxi records from the New York City Taxi and Limousine Commission (NYC TLC) [31]. The checksum-pinned Parquet file contains 2,964,624 rows; a dispersed affine permutation selects nested subsets of 10,000, 100,000 and 1,000,000 rows using seed 20261001. We retain eleven fields: vendor, pickup/drop-off times, passenger count, distance, pickup/drop-off locations, payment type, fare, tip and total. Keys combine the release, file-checksum prefix and physical row ordinal, and therefore identify rows within this frozen file rather than trips across releases. Compact sorted-key UTF-8 JSON preserves values; nulls remain null. The million-row subset has 47,277 missing passenger counts and no serialization failures. Identical field values remain separate rows. The affine sample is not a uniformly random permutation.

The Linux campaign evaluates the 100,000-row subset with ten commits and 100 corrections per commit; the million-row diagnostic uses ten commits and 1,000 corrections per commit. Each correction increases fare and total by 0.01 while preserving the other retained fields. All systems receive identical records and mutations. These are generated histories over public records, not observed longitudinal edits. The inherited `payload_bytes=8` configuration field is unused by the public-data builder; public values retain the eleven fields and are accounted for by their measured serialized bytes. The harness stores each canonical record as a JSON-encoded string in the common value

column. These timings do not characterize native nested-object mutation or field-level query performance.

Table 4 Checksum-pinned public data and Linux protocol

Item	Definition
Source	NYC TLC yellow_tripdata_2024-01.parquet; 2,964,624 source rows
Raw SHA-256	c4d59da7bbc8abaeceb1727947ee93d9891a71acb42854bd80db1571b2030510
Prepared million-row SHA-256	6608b7680e82432ca22f90a71d4994676ffd290c4fc85f17f91fab03d4c7b661
P1	100,000 records; H=10; C=100; 2 warm-ups + 7 measured trials per system
Scale diagnostic	1,000,000 records; H=10; C=1,000; one trial per system; no interval or ranking

Timing, correctness and uncertainty

Every primary execution uses a fresh worker and a new repository. Model order is deterministically shuffled and rotated within repetition blocks and retained in the protocol. Import timing reaches the first durable version. Incremental-commit latency is the median of the history's commit timers. Diff compares the first and last versions and includes sorted changed keys plus old/new value hashes serialized as canonical JSON. Checkout materializes the full sorted key/value state as canonical UTF-8 JSON. Dolt timings include CLI launch, JSON parsing and output normalization; Revon operates through its in-process API. No engine-only speed claim follows from this comparison.

Outside the operation timers, every primary execution checks initial and final state hashes, the observed canonical semantic diff against an independently constructed workload oracle, and all H+1 historical states. Verification JSON retains observed and expected hashes, not only expected key counts. The post-run audit regenerates workloads, checks the exact matrix and selected paths, verifies source/data/protocol identity, and recomputes all summaries. A second local audit checks the downloaded evidence. These are internal integrity checks; they do not constitute independent timing reproduction.

We report medians and interquartile ranges (IQRs) from seven measured repository trials, with warm-ups excluded. Matched-trial latency ratios are summarized by their median and a paired percentile bootstrap interval using 20,000 resamples. Ratios above one favour the denominator. These exploratory 95% intervals describe repeated runs of a fixed workload on one VM; they are not population confidence intervals and are not adjusted for multiple comparisons. A median ratio need not equal the ratio of two latency medians. All completed trials, including outlying timings, remain in the evidence.

The parent samples worker and descendant RSS, CPU and I/O counters every 10 ms. Whole-worker peaks include workload generation, oracle state and verification, not solely the storage engine. Short-lived subprocess peaks or counters may be missed. The original

sampler reports an all-zero read counter as missing, so no primary read-I/O claim is made. The separate supplement distinguishes observed zero from unavailable counters; it is not retroactively merged into the primary data. Physical read bytes can be zero for cached workloads and do not measure logical bytes examined.

Repository footprint is the directory size before close and compaction, including indexes, metadata and representation costs. We also retain logical payload normalization and selected first-trial compaction probes. These are workflow measures, not a decomposition into pure tree overhead. CPU, RSS and I/O cover the complete worker lifecycle; operation timers and resource counters therefore have different scopes.

Frozen threshold and geometry controls

The earlier Windows threshold study tested 1,024, 2,048, 4,096, 8,192 and 16,384 operations using predefined, disjoint calibration and validation profiles. It selected $T=16,384$ before validation by minimizing normalized calibration latency. The held-out normalized score was 1.032, compared with 1.297 for $T=4,096$; always-log scored 1.000 on both splits. These historical cached-query scores remain tied to their archived source and environment, not the repaired Linux implementation. The largest tested candidate winning does not establish an optimum. The earlier long-history and three-seed pilots remain exploratory supplementary evidence.

The Linux threshold control freezes $T=16,384$ and tests 10,000 rows with 512 changes per commit at $H=16$ and $H=64$, giving 8,192 and 32,768 accumulated operations. Forced log, forced Merkle and the hybrid run in separate workers with two warm-ups and seven measured trials. No Linux result is used to alter the threshold. This is a two-case transport check of the existing choice, not a new calibration/validation cycle.

The geometry supplement uses one spread workload with 10,000 rows, ten commits, 100 changes per commit, 32-byte synthetic values and seed 20260824. It fixes forced Merkle and tests b4-d6, b8-d3, b8-d4, b8-d5 and b16-d3. Each geometry has two warm-ups and seven measured trials in a fresh worker. A rotating order limits configuration/time confounding. Every execution checks all eleven historical states and the endpoint diff outside timing. Geometry timings use the same adapter output contract, but resource peaks from its inherited in-process monitor are not pooled with external RSS measurements.

5 Linux Results

Hybrid versus forced Merkle

Revon-M and Revon-H retain the same persistent representation and changesets; the ablation changes the requested diff strategy. Figure 3 and Table 5 show all nine repeated-trial comparisons. The hybrid interval excludes parity in eight scenarios; the dense S3 interval includes one. Sparse and repeated-key results support access to log aggregation under the tested conditions. They do not establish a universal advantage from switching or a representation-level storage saving.

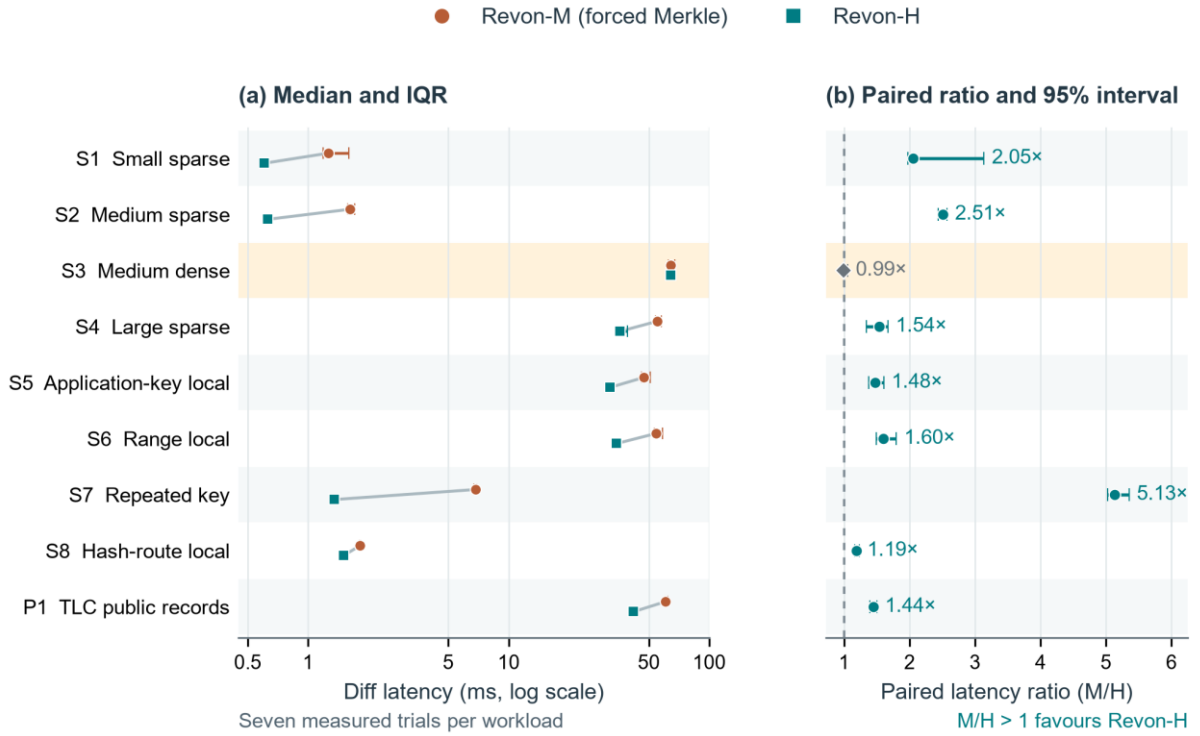


Fig. 3 Linux Revon-M and Revon-H diff comparison. (a) Median latency with interquartile ranges on a logarithmic axis; paired medians are connected within each workload. (b) Median matched-trial M/H latency ratio with paired percentile bootstrap 95% intervals (20,000 resamples); the dashed line denotes parity. Each workload has seven measured trials. The highlighted S3 dense case has an interval containing parity; the other eight favour H. Ratios are medians of matched-trial ratios, not ratios of the two latency medians. Workload definitions are in Table 3. These intervals concern repeated timings for fixed histories on one VM, not independent datasets or machines.

Table 5 Paired M/H diff ablation; milliseconds and median trial ratio [95% interval]

ID	M (ms)	H (ms)	M/H ratio [interval]	H path
S1	1.259	0.604	2.05 [1.97, 3.14]	log
S2	1.620	0.629	2.51 [2.44, 2.57]	log
S3	64.135	64.183	0.99 [0.97, 1.06]	merkle
S4	54.897	35.749	1.54 [1.34, 1.67]	log
S5	47.253	31.967	1.48 [1.38, 1.60]	log
S6	54.352	34.537	1.60 [1.50, 1.80]	log
S7	6.862	1.345	5.13 [5.02, 5.35]	log
S8	1.817	1.503	1.19 [1.17, 1.23]	log
P1	60.597	41.964	1.44 [1.39, 1.50]	log

For P1, Revon-H has a 41.964 ms diff median versus 60.597 ms for forced Merkle. Both use the same eleven-field public records and generated updates. S3 accumulates 10,000 operations and uses Merkle in H; its near-parity result is consistent with both variants executing the same diff path plus selection work.

Threshold control and adverse result

Figure 4 and Table 6 report both threshold workloads, including the result adverse to the frozen policy. At 8,192 operations, H uses log. At 32,768 operations, H selects Merkle, although forced log is faster in all seven matched measured trials. The operation count alone is insufficient to select the fastest path for these cases. The Linux control therefore completes the validation task while leaving the threshold optimality question unresolved.

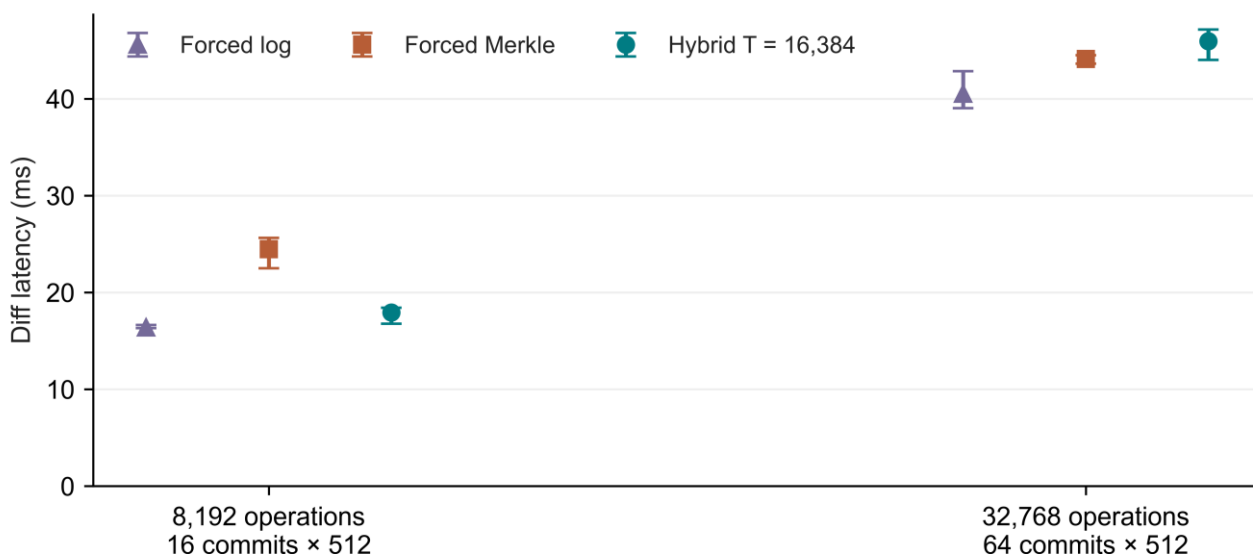


Fig. 4 Linux control for the previously frozen T=16,384 policy; medians and IQRs from seven measured trials per path, with different history lengths labelled explicitly

Table 6 Frozen threshold controls; diff medians in milliseconds

Operations / history	Forced log	Forced Merkle	Hybrid	Selected path
8,192 / H=16	16.364	24.466	17.926	log
32,768 / H=64	40.520	44.120	45.944	merkle

Complete workflow comparison

Table 7 compares canonical diff outputs from the Revon-H API and Dolt SQL/CLI workflows. All nine paired Dolt/H intervals exceed one. This is evidence for these complete workflows; process startup, language, serialization, query execution and resident state are confounded with the data structure. Figure 5 reports commit and checkout separately so the diff result is not generalized to every operation.

Table 7 Dolt SQL/CLI versus Revon-H diff; milliseconds and paired ratio [95% interval]

ID	Dolt (ms)	H (ms)	Dolt/H ratio [interval]
S1	90.704	0.604	148.15 [139.77, 160.15]
S2	95.515	0.629	148.70 [140.08, 150.80]
S3	297.617	64.183	4.70 [4.54, 4.77]
S4	152.472	35.749	4.25 [3.84, 4.34]
S5	125.594	31.967	3.73 [3.71, 3.87]
S6	135.765	34.537	3.88 [3.69, 4.12]
S7	97.451	1.345	71.73 [68.95, 74.32]
S8	110.966	1.503	71.24 [68.83, 77.06]
P1	168.576	41.964	4.02 [3.77, 4.07]

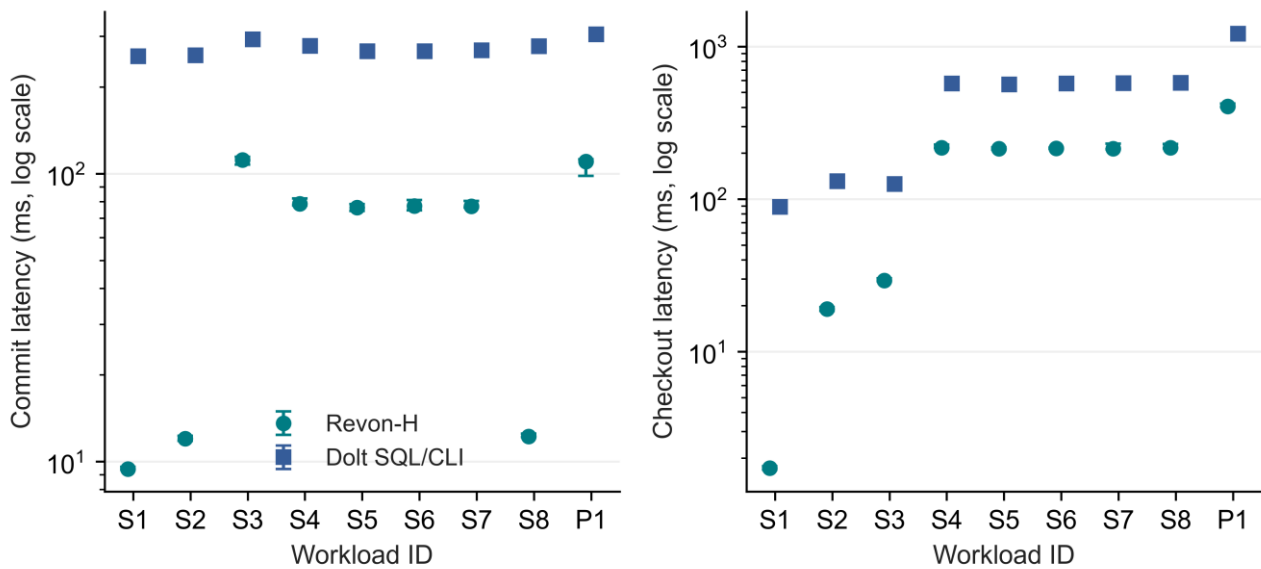


Fig. 5 Linux incremental-commit and full-checkout latency for Revon-H and Dolt SQL/CLI; points show medians and bars show IQRs of seven trials, using logarithmic latency

Storage and memory costs

Figure 6 and Table 8 retain repository footprint and complete-worker RSS alongside latency. M and H both store addressed changesets. Any small directory-size difference between them is a measured workflow difference, not evidence that selecting a log path removes trie data. Dolt uses different serialization, compression and indexing. A low diff latency can coexist with a substantially larger retained repository; the resource costs constrain the scope of the efficiency claim.

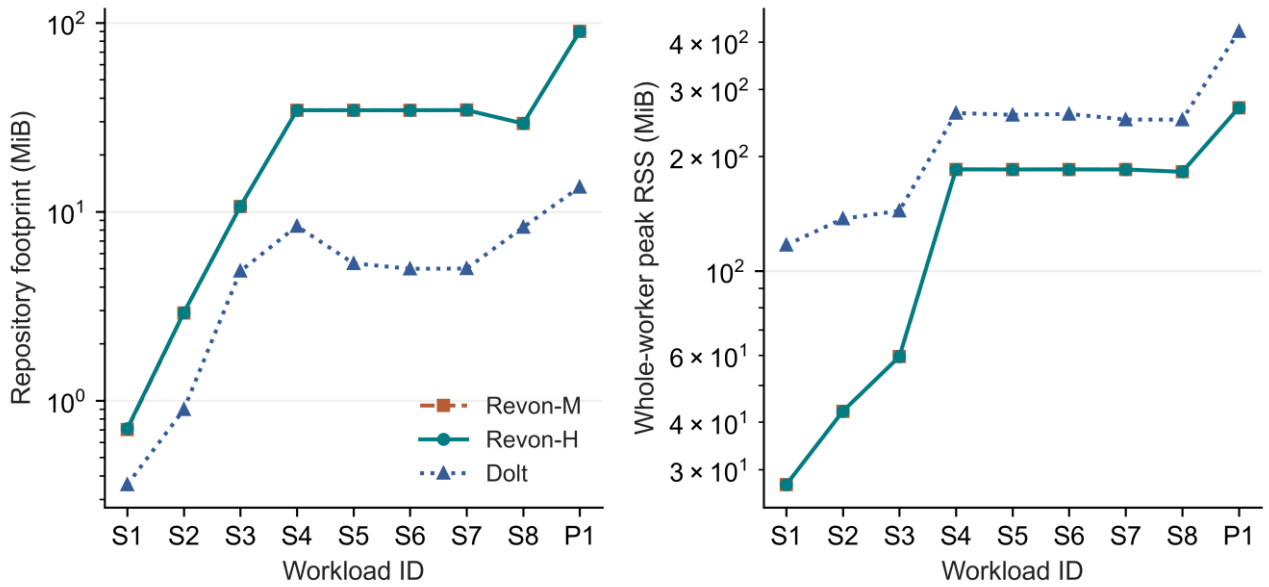


Fig. 6 Median repository directory size before compaction and median whole-worker peak RSS across seven Linux trials; logarithmic scales, including shared workload and verification memory in RSS

Table 8 Repository footprint and worker RSS; medians in MiB

ID	M disk	H disk	Dolt disk	H RSS	Dolt RSS
S1	0.7	0.7	0.4	27.4	116.9
S2	2.9	2.9	0.9	42.7	137.1
S3	10.7	10.7	4.8	59.5	143.9
S4	34.4	34.4	8.4	184.9	260.1
S5	34.4	34.4	5.3	184.8	257.2
S6	34.4	34.4	5.0	184.9	258.6
S7	34.5	34.5	5.0	184.8	250.0
S8	29.3	29.3	8.3	182.2	249.9
P1	90.1	90.0	13.5	268.6	426.5

Million-record feasibility

The million-row diagnostic verifies one fresh repository for each of M, H and Dolt, including all eleven historical states and the common diff oracle. Table 9 reports these observations without IQRs, confidence intervals or a performance ranking. H uses Merkle because the 10,000 accumulated operations exceed $T=4,096$. These runs establish that the bounded workflow completed on this VM, not reliable relative scaling or an optimal configuration. Repeated million-row trials remain future work.

Table 9 Single-run million-record diagnostics; n=1 per system, no comparative inference

System	Import (s)	Commit (ms)	Diff (ms)	Checkout (s)	RSS (GiB)
Dolt	24.902	547.311	817.034	11.454	2.555
Revon-H	32.289	2961.901	1233.601	4.729	2.561
Revon-M	32.454	2979.717	1265.367	4.685	2.561

Fixed-trie geometry and telemetry

Figure 7 and Table 10 describe the fresh geometry supplement. The equal-capacity b4-d6, b8-d4 and b16-d3 configurations each have 4,096 leaf buckets and equal expected occupancy; their contrast concerns routing shape. Increasing depth at b=8 changes bucket capacity and the traversal/leaf-scan trade-off. One workload and one VM do not establish a universal geometry optimum.

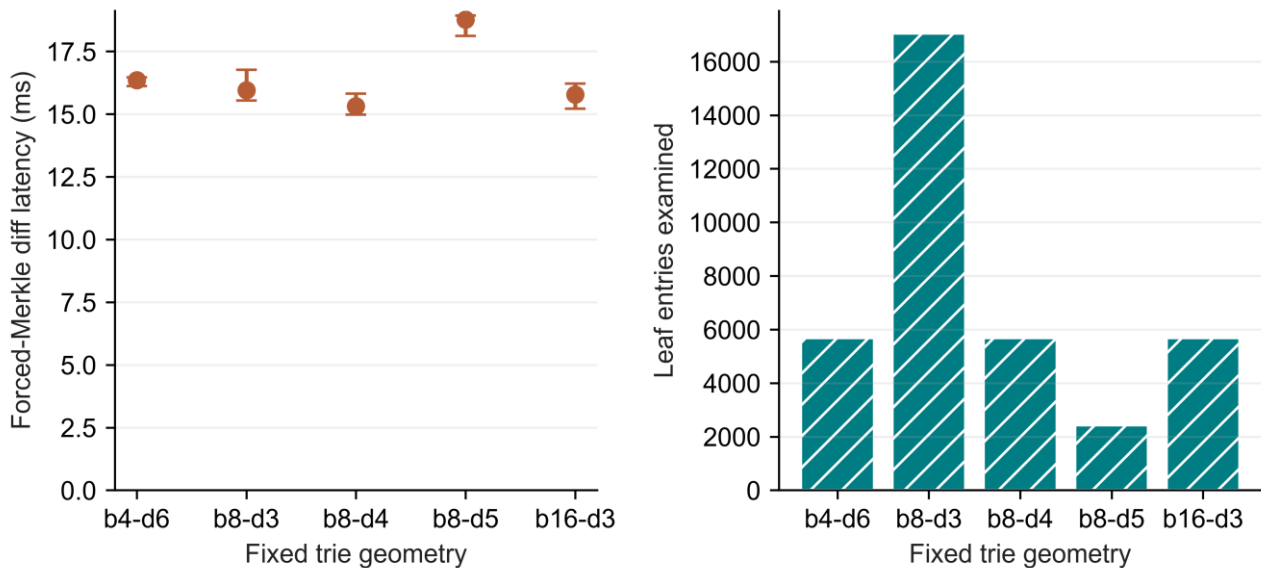


Fig. 7 Linux fixed-trie geometry supplement; left shows forced-Merkle diff medians and IQRs from seven trials, right shows median examined leaf entries

Table 10 Geometry work and footprint; one fixed spread workload

Geometry	Buckets	Diff (ms)	Leaf entries	Node pairs	Disk (MiB)
b4-d6	4096	16.355	5682	3628	4.32
b8-d3	512	15.949	17048	585	6.88
b8-d4	4096	15.316	5682	3729	4.25
b8-d5	32768	18.765	2426	6089	6.18
b16-d3	4096	15.775	5682	3929	8.12

The separate telemetry regression exercises valid all-zero I/O samples and unavailable counters through real isolated workers. Both tests pass. The 27-execution Linux

supplement then checks M, H and Dolt on a fixed 10,000-row workload with two warm-ups and seven measured trials per system. These rows retain zero read bytes as observations rather than blanks. They validate counter handling; they do not establish cold-storage behavior, eliminate sampling loss or alter the primary latency summaries.

Representation baselines

Table 11 provides the complete six-workflow comparison for the representative S4 spread workload. These medians are descriptive; all eight synthetic scenarios, trial dispersion, normalized storage and the separate SQL/CSV import variants are retained in Online Resource 1. Snapshot and Log-only isolate representation choices within the common harness, not production feature equivalence. Their results make the cost of retaining full states or replaying operations visible alongside Revon.

Table 11 Representative S4 workflow medians; complete eight-scenario matrix in Online Resource 1

Workflow	Import (s)	Commit (ms)	Diff (ms)	Checkout (ms)	Disk (MiB)
Snapshot	0.071	69.577	624.860	168.158	53.5
Log-only	0.072	2.474	16.483	159.538	5.0
Revon-M	1.488	78.842	54.897	215.661	34.4
Revon-H	1.644	78.590	35.749	217.482	34.4
Dolt SQL	1.663	277.593	152.472	572.585	8.4
Dolt CSV	1.335	273.425	149.325	559.342	7.9

6 Discussion

RQ1: the Linux evidence supports selected diff-workflow advantages and documents storage, memory and commit costs. Snapshot, Log-only and both Dolt import paths remain in the complete raw matrix even where figures emphasize the central M/H/Dolt comparison. The comparison is over a bounded single-writer versioning workflow, with common materialized outputs. It does not establish superior SQL query processing, transaction concurrency, relational capabilities or general structured-data management.

RQ2: access to log aggregation improves on forced Merkle in eight of nine paired repeated-trial cases, with one inconclusive dense case. Always-log remains a necessary static control. The frozen threshold's adverse Linux result shows that a successful M/H ablation does not validate adaptation itself. A richer selector should be trained on fresh calibration histories and tested on untouched validation histories; these results are not used to implement or tune one.

RQ3: the geometry supplement supplies current-source Linux work counters and timings under a rotated execution order. Expected bucket occupancy explains only part of the cost: touched leaf serialization, ancestor reconstruction, output size and traversal shape also

matter. Equal-capacity configurations must not be described as changing expected occupancy.

The JSON repair is part of all current Linux experiments. Earlier API defects allowed retained mutable aliases and Python-equality collisions, including true versus 1 and 1 versus 1.0. Input/output detachment and v1 encoded-identity comparisons repair these semantics without changing the existing object encoding or on-disk schema. Historical pre-repair timings remain archived and are not relabelled. The Linux trials demonstrate repaired-source behavior on the supplied histories; unit tests additionally cover null/absence, nested ownership, unsupported values and rollback.

7 Limitations And Future Work

External validity: the new cloud VM is separate from the original Windows/WSL host, but all Linux repetitions use one hardware configuration and are author-run. Hardware, operating system, interpreter build, virtualization and storage change together. We cannot attribute differences solely to Linux or infer performance across machines. Google Ops Agent and filesystem caching remain active; neither a perfectly idle machine nor cold I/O is claimed.

Workload validity: the synthetic scenarios use one deterministic history per scenario. TLC supplies public record contents but generated corrections, one monthly release and nested affine samples. It supplies no natural edit history, relational queries, multi-table joins or diverse public datasets. Million-row Linux evidence has one execution per system and establishes feasibility only. Additional seeds, observed histories, datasets and repeated scale trials are required for broader inference.

Implementation scope: the trie geometry is fixed and the repository eagerly loads retained nodes, changesets and commits into memory. It has no eviction policy. Revon supports linear history and a single writer; named branches, merge commits, conflict handling, schema evolution, remotes, ordered scans, crash fault injection and concurrent workloads are not evaluated. Atomic SQLite transactions do not prove all production durability or operational guarantees.

Measurement and inference: API/CLI boundaries and resident-state asymmetry preclude a storage-engine speed claim. Ten-millisecond sampling may miss short peaks and child counters. Whole-worker RSS contains shared oracle data and cannot be read as product-only memory. Keys, log operations and trie-node pairs are different work units, and Dolt internal work is not exposed. Repetition intervals describe fixed histories, with no multiple-comparison correction. Source/hash audits check internal integrity, not independent performance reproduction.

Selection: T=16,384 was the largest candidate in a bounded historical search; the cloud controls neither locate a crossover nor prove an optimum. Its slower choice at 32,768 operations must be retained. Future work should hold row count and history length constant, vary locality and density independently, fix process/hash-seed conditions and use independent histories as the uncertainty unit. A new policy needs separate calibration and held-out validation before replacing the frozen implementation.

Storage improvements may include binary nodes, value chunks, packed objects, compression and reachability-based garbage collection. An ordered Merkle index or content-defined chunking could support range operations. These changes require separate

semantic and performance studies rather than being inferred from the present fixed-trie results.

8 Conclusion

Revon combines incremental fixed-trie updates, addressed changesets and atomic SQLite persistence with two semantically consistent version-diff paths. The repaired source passed 516 Linux workflow executions and a separate 72-execution geometry/telemetry supplement on a cloud machine. Repeated M/H comparisons support log-path benefits for selected workloads, while the dense interval includes parity. The frozen threshold can select a slower path than forced log, so the study does not establish optimal adaptation. Single-run million-record checks demonstrate bounded feasibility and expose resource costs without a comparative scaling claim. The contribution is an auditable prototype and trade-off study over complete versioning workflows, with broader generality and independent reproduction still open.

APPENDIX A. REPRODUCIBILITY

Online Resource 1 (Revon_Linux_Reproducibility_Supplement.zip) contains the exact two executed source snapshots, frozen protocols, manifests, raw rows, paired intervals, per-trial semantic verification, audit records and geometry observations. The primary directory is linux-validation-20261008 and the additional directory is linux-final-supplement-20261008. They are separate experiments and their timing rows must not be pooled. README instructions describe data acquisition, pinned environment setup, commands and internal verification. TLC records and generated repositories are rebuilt instead of redistributed. This follows the principle of retaining a complete computational provenance chain [24].

For the primary snapshot, run `python -m experiments.linux_validation --output <new-directory> --dataset <prepared-TLC-directory> with PYTHONHASHSEED=20261008`, pinned Python 3.14.3 and Dolt 2.3.1. For the second snapshot, create tests/test_data, run the 69-test suite, then `python -m experiments.linux_final_supplement --output <new-directory>`. Environment gates and unit tests run before measurement. Archived audits can be rerun from their corresponding source directories; public metadata and records must match the retained dataset.json. Raw timing reproduction will vary with environment even if semantic hashes agree.

DECLARATIONS

Funding: The authors received no funding for this work.

Competing interests: The authors declare no competing interests.

Author contributions: Atharva Sheersh Pandey carried out 80% of the research work and Adhyan Jain 20%. Poornima Nedunchezian provided guidance and critical review.

Ethics and consent to participate: No participants were recruited and no human or animal intervention was performed. This computational study uses synthetic histories and publicly released de-identified TLC records. The study does not claim institutional ethics approval or exemption.

Consent for publication: Not applicable; no identifiable participant material is included.

Data and code availability: The TLC source file is publicly accessible through NYC TLC [31]; the retained checksum and preparation procedure identify the exact input. The source snapshots and experimental evidence supporting the Linux results are supplied as Online Resource 1 with this manuscript. The public project repository is <https://github.com/atharvasheersh/Revon>; historical evidence is pinned at commit 37d27d465ffc922a3881b85bb415574f956a9a09. The new supplementary package is supplied with the manuscript and is not claimed to be present at that historical commit.

ACKNOWLEDGMENT

We thank the open-source Dolt project and the researchers whose work on dataset versioning informed our experimental design.

REFERENCES

- [1] S. Chacon and B. Straub, “Git Internals: Git Objects,” in *Pro Git*, 2nd ed., 2014. [Online]. Available: <https://git-scm.com/book/en/v2/Git-Internals-Git-Objects>.
- [2] DoltHub, “Dolt Storage Engine: Block Store,” Dolt Documentation, accessed Oct. 2, 2026. [Online]. Available: <https://www.dolthub.com/docs/architecture/storage-engine/block-store/>.
- [3] T. Sehn, “Dolt’s Storage Engine,” DoltHub Blog, Feb. 29, 2024. [Online]. Available: <https://www.dolthub.com/blog/2024-02-29-storage-engine/>.
- [4] R. C. Merkle, “A digital signature based on a conventional encryption function,” in *Advances in Cryptology: CRYPTO ’87, Lecture Notes in Computer Science*, vol. 293, 1988, pp. 369-378, https://doi.org/10.1007/3-540-48184-2_32.
- [5] J. R. Driscoll, N. Sarnak, D. D. Sleator, and R. E. Tarjan, “Making data structures persistent,” *J. Comput. Syst. Sci.*, vol. 38, no. 1, pp. 86-124, 1989, [https://doi.org/10.1016/0022-0000\(89\)90034-2](https://doi.org/10.1016/0022-0000(89)90034-2).
- [6] J. Benet, “IPFS: Content Addressed, Versioned, P2P File System,” arXiv:1407.3561, 2014. [Online]. Available: <https://arxiv.org/abs/1407.3561>.
- [7] A. Auvolet and F. Taïani, “Merkle Search Trees: Efficient State-Based CRDTs in Open Networks,” in *Proc. 38th IEEE Int. Symp. Reliable Distributed Syst. (SRDS)*, 2019, pp. 221-230, <https://doi.org/10.1109/SRDS47363.2019.00032>.
- [8] A. Muthitacharoen, B. Chen, and D. Mazières, “A low-bandwidth network file system,” in *Proc. 18th ACM Symp. Operating Syst. Principles (SOSP)*, 2001, pp. 174-187, <https://doi.org/10.1145/502034.502052>.
- [9] A. Bhardwaj et al., “DataHub: Collaborative Data Science and Dataset Version Management at Scale,” in *Proc. 7th Biennial Conf. Innovative Data Syst. Research (CIDR)*, 2015. Open copy: https://www.cidrdb.org/cidr2015/Papers/CIDR15_Paper18.pdf.
- [10] M. Maddox et al., “Decibel: The Relational Dataset Branching System,” *Proc. VLDB Endowment*, vol. 9, no. 9, pp. 624-635, 2016, <https://doi.org/10.14778/2947618.2947619>.

- [11] S. Huang, L. Xu, J. Liu, A. J. Elmore, and A. Parameswaran, "OrpheusDB: Bolt-on Versioning for Relational Databases," *Proc. VLDB Endowment*, vol. 10, no. 10, pp. 1130-1141, 2017, <https://doi.org/10.14778/3115404.3115417>.
- [12] S. Bhattacharjee, A. Chavan, S. Huang, A. Deshpande, and A. Parameswaran, "Principles of Dataset Versioning: Exploring the Recreation/Storage Tradeoff," *Proc. VLDB Endowment*, vol. 8, no. 12, pp. 1346-1357, 2015, <https://doi.org/10.14778/2824032.2824035>.
- [13] M. Armbrust et al., "Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores," *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 3411-3424, 2020, <https://doi.org/10.14778/3415478.3415560>.
- [14] Attic Labs, "Noms Technical Overview." GitHub. [Online]. Available: <https://github.com/attic-labs/noms/blob/master/doc/intro.md>.
- [15] S. Wang et al., "ForkBase: An Efficient Storage Engine for Blockchain and Forkable Applications," *Proc. VLDB Endowment*, vol. 11, no. 10, pp. 1137-1150, 2018, <https://doi.org/10.14778/3231751.3231762>.
- [16] A. Twigg, A. Byde, G. Milos, T. Moreton, J. Wilkes, and T. Wilkie, "Stratified B-trees and versioning dictionaries," in *Proc. 3rd USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage)*, 2011. [Online]. Available: <https://arxiv.org/abs/1103.4282>.
- [17] A. Seering, P. Cudré-Mauroux, S. Madden, and M. Stonebraker, "Efficient versioning for scientific array databases," in *Proc. IEEE 28th Int. Conf. Data Engineering (ICDE)*, 2012, pp. 1013-1024, <https://doi.org/10.1109/ICDE.2012.102>. [Online]. Available: <https://dspace.mit.edu/entities/publication/a5c2c050-6abd-47c1-8bee-4f8c51b6a15f>.
- [18] E. Soroush and M. Balazinska, "Time travel in a scientific array database," in *Proc. IEEE 29th Int. Conf. Data Engineering (ICDE)*, 2013, pp. 98-109, <https://doi.org/10.1109/ICDE.2013.6544817>. [Online]. Available: https://scidb.cs.washington.edu/paper/ICDE13_conf_full_422.pdf.
- [19] G. S. Brodal, S. Sioutas, K. Tsakalidis, and K. Tsichlas, "Fully persistent B-trees," *Theoretical Computer Science*, vol. 841, pp. 10-26, 2020, <https://doi.org/10.1016/j.tcs.2020.06.027>. [Online]. Available: <https://cs.au.dk/~gerth/papers/tcs20.pdf>.
- [20] N. Saquib, C. Krintz, and R. Wolski, "Replicated versioned data structures for wide-area distributed systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 1, pp. 207-224, 2023, <https://doi.org/10.1109/TPDS.2022.3217969>. [Online]. Available: <https://racelab.cs.ucsb.edu/papers/tpds22-versioned.pdf>.
- [21] M. E. Schüle, L. Karnowski, J. Schmeißer, B. Kleiner, A. Kemper, and T. Neumann, "Versioning in main-memory database systems: From MusaeusDB to TardisDB," in *Proc. 31st Int. Conf. Scientific and Statistical Database Management (SSDBM)*, 2019, pp. 169-180, <https://doi.org/10.1145/3335783.3335792>. [Online]. Available: <https://www-db.cs.tum.edu/~schuele/data/tardis.pdf>.
- [22] P. Buneman, S. Khanna, and W.-C. Tan, "Why and where: A characterization of data provenance," in *Database Theory: ICDT 2001, Lecture Notes in Computer Science*, vol.

- 1973, 2001, pp. 316-330, https://doi.org/10.1007/3-540-44503-X_20. [Online]. Available: https://www.pure.ed.ac.uk/ws/files/16509989/Why_and_Where_A_Characterization_of_Data_Provenance.pdf.
- [23] F. McSherry, D. G. Murray, R. Isaacs, and M. Isard, "Differential dataflow," in Proc. 6th Biennial Conf. Innovative Data Systems Research (CIDR), 2013. [Online]. Available: https://www.cidrdb.org/cidr2013/Papers/CIDR13_Paper111.pdf.
- [24] M. Akhlaghi et al., "Toward long-term and archivable reproducibility," Computing in Science & Engineering, vol. 23, no. 3, pp. 82-91, 2021, <https://doi.org/10.1109/MCSE.2021.3072860>. [Online]. Available: <https://arxiv.org/abs/2006.03018>.
- [25] R. Shraga and R. J. Miller, "Explaining Dataset Changes for Semantic Data Versioning with Explain-Da-V," Proc. VLDB Endow., vol. 16, no. 6, pp. 1587-1600, 2023. <https://doi.org/10.14778/3583140.3583169>.
- [26] A. Guo, J. Li, P. Sukprasert, S. Khuller, A. Deshpande, and K. Mukherjee, "To store or not to store: A graph theoretical approach for dataset versioning," in Proc. IEEE Int. Parallel and Distributed Processing Symposium (IPDPS), 2024, pp. 479-493, <https://doi.org/10.1109/IPDPS57955.2024.00049>. [Online]. Available: <https://arxiv.org/abs/2402.11741>.
- [27] A. González-Cebrián, M. Bradford, A. E. Chis, and H. González-Vélez, "Standardised versioning of datasets: A FAIR-compliant proposal," Scientific Data, vol. 11, art. 358, 2024, <https://doi.org/10.1038/s41597-024-03153-y>. [Online]. Available: <https://www.nature.com/articles/s41597-024-03153-y>.
- [28] J. Zhao and S. Krishnan, "Compression and in-situ query processing for fine-grained array lineage," arXiv:2405.17701, 2024. [Online]. Available: <https://arxiv.org/abs/2405.17701>.
- [29] H. Gou, Z. Zhang, Y. Sun, P. Xu, F. Tian, L. Wang, and J. Wang, "Git4Data: Database-native version control for AI agents," arXiv:2609.02106, 2026, preprint. [Online]. Available: <https://arxiv.org/abs/2609.02106>.
- [30] J. Edwards and T. Petricek, "Baseline: Operation-based evolution and versioning of data," arXiv:2512.09762, 2025, preprint, updated 2026. [Online]. Available: <https://arxiv.org/abs/2512.09762>.
- [31] New York City Taxi and Limousine Commission, "TLC Trip Record Data: Yellow Taxi, January 2024," public dataset, accessed Oct. 1, 2026. [Online]. Available: <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>.